**UCS 503 – Software Engineering**

Mid-Semester Evaluation

---

# EventPulse

## Predictive Event Operations Platform

---

**Group: 3P12**


| Team Member | Roll Number |
|---|---|
| **Arpit Goswami** | 1024160134 |
| **Anushka Verma** | 1026160034 |
| **Rajat Sharma** | 1024160049 |

**BE Third Year, CSE**


**Submitted To**

Sandeep Kaur


**CSE Department**

Thapar Institute of Engineering and Technology, Patiala

# Contents

**1 Introduction** **1**
1.1 Background and Context . . . . . 1
1.1.1 Problem Statement . . . . . 1
1.2 Project Objectives . . . . . 1

**2 Methodology and Design** **1**
2.1 System Architecture . . . . . 1
2.1.1 High-Level Design . . . . . 1
2.1.2 Technical Stack . . . . . 2
2.2 Implementation Details . . . . . 2
2.2.1 Module 1: Core Functionality . . . . . 2
2.2.2 Module 2: Secondary Features . . . . . 2

**3 Results and Evaluation** **3**
3.1 Testing Methodology . . . . . 3
3.2 Performance Metrics . . . . . 4
3.3 Analysis . . . . . 4

**4 Conclusion** **4**
4.1 Summary of Work . . . . . 4
4.2 Key Findings . . . . . 4
4.3 Future Work and Recommendations . . . . . 4
4.4 Final Remarks . . . . . 4

**Bibliography** **5**

# 1 Introduction

## 1.1 Background and Context

Large events (fests, workshops, conferences, hackathons) are usually run through disconnected tools such as spreadsheets, forms, messaging groups, and manual attendance records. This causes entry delays, inaccurate attendance data, poor volunteer coordination, and slow incident handling. **EventPulse** is a unified, predictive event operations platform that combines event planning, registration, secure QR ticketing, check-in, ticket transfer, volunteer coordination, live monitoring, incident and resource management, and post-event reporting.

### 1.1.1 Problem Statement

1. **Fragmented operations:** planning, attendance, volunteers, and incidents sit in separate systems.
2. **Entry challenges:** manual checks and static QR codes cause delays and duplicate or unauthorized entry.
3. **Volunteer coordination:** role, shift, and availability tracking is manual.
4. **Limited visibility:** no central real-time view of gates, attendance, and incidents.
5. **No decision support:** event data is not turned into insights on peaks, congestion, or demand.

## 1.2 Project Objectives

1. **Unified management:** event creation, venue setup, capacity, and ticket categories.
2. **Secure check-in:** unique QR tickets validated server-side.
3. **Ticket transfer:** old QR invalidated and a new ticket issued.
4. **Volunteer coordination:** registration, roles, shifts, and attendance.
5. **Live monitoring:** attendance, gate activity, and incident visibility.
6. **Operational support:** incidents, resources, alerts, and reminders.
7. **Predictive support:** arrival peaks, congestion, and demand estimates.
8. **Post-event analysis:** feedback, certificates, and reports.

# 2 Methodology and Design

## 2.1 System Architecture

EventPulse uses a modular client–server architecture; the backend is the central authority for authentication, authorization, business rules, ticket validation, and attendance. It has four layers: **Client** (attendee, organizer, volunteer, scanner interfaces), **Application** (event, ticketing, QR validation, volunteer, and incident logic), **Data and Processing** (users, events, tickets, attendance, incidents), and **Analytics** (predictive outputs for operational decisions).

### 2.1.1 High-Level Design

Users access the React frontend; requests go to the NestJS backend, which communicates with the PostgreSQL database and the analytics service. Four role-based interfaces are provided: **Attendee** (registration, tickets, transfer, feedback), **Organizer** (events, monitoring, volunteers, incidents, reports), **Volunteer** (roles, shifts, attendance), and **Scanner** (QR validation and entry recording).

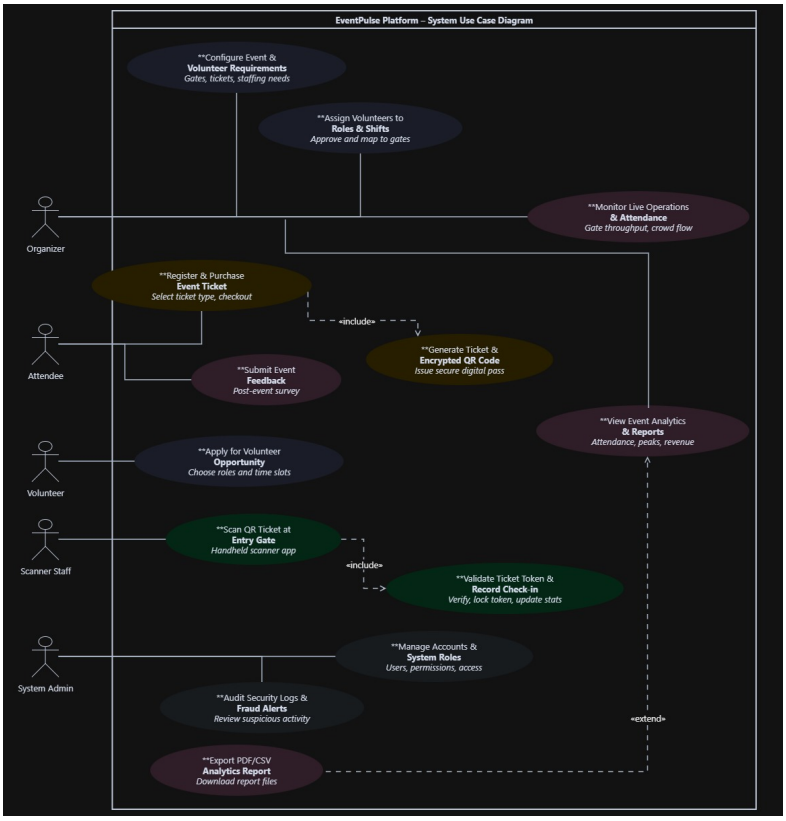



**Figure 2.1:** EventPulse System Use Case Diagram

### 2.1.2 Technical Stack

**Table 2.1:** Technology Stack and Current Status

| Component | Technology | Status |
|---|---|---|
| Frontend | React.js, TypeScript, Tailwind CSS | Completed |
| Backend | Node.js, NestJS | In Development |
| Database | PostgreSQL | Integration / Planned |
| Real-Time | Socket.IO | Planned |
| Background Jobs | Redis, BullMQ | Planned |
| Analytics | Python, FastAPI, pandas, scikit-learn | Planned / In Development |
| Offline Scanner | SQLite | Future Extension |
| Version Control / CI | Git, GitHub, GitHub Actions | In Use / Planned |

## 2.2 Implementation Details

### 2.2.1 Module 1: Core Functionality

**Event setup and registration.** Organizers configure event details, schedule, venue, capacity, and ticket categories. Attendees register against an event and ticket category and receive a digital ticket with a unique QR code that represents the ticket rather than authorizing entry by itself.

**Server-side QR validation.** At the gate, the backend checks that the ticket exists, belongs to the event, is valid, is not expired or cancelled, and has not been used. A valid ticket moves from `ISSUED` to `CHECKED_IN`; duplicate scans are rejected.

**Ticket transfer.** The old ticket is invalidated and a new one with a new QR code is issued to the recipient.

**Frontend.** Attendee ticketing, organizer panels, and scanner views are complete; backend and database integration are in progress.

### 2.2.2 Module 2: Secondary Features

- **Volunteer Hub:** applications, role and shift assignment, attendance, and temporary opportunities.

- **Live Operations:** dashboard for attendance, gate activity, volunteers, incidents, and resources (Socket.IO updates planned).
- **Incidents and Resources:** incident reporting for organizers, resource tracking, and shortage alerts.
- **Notifications:** confirmations, reminders, shift alerts, and transfer notices.
- **Predictive Analytics:** arrival peaks, gate congestion, volunteer and resource demand; current experiments use simulated data and show feasibility only.
- **Post-Event:** feedback, certificates, and summary reports.

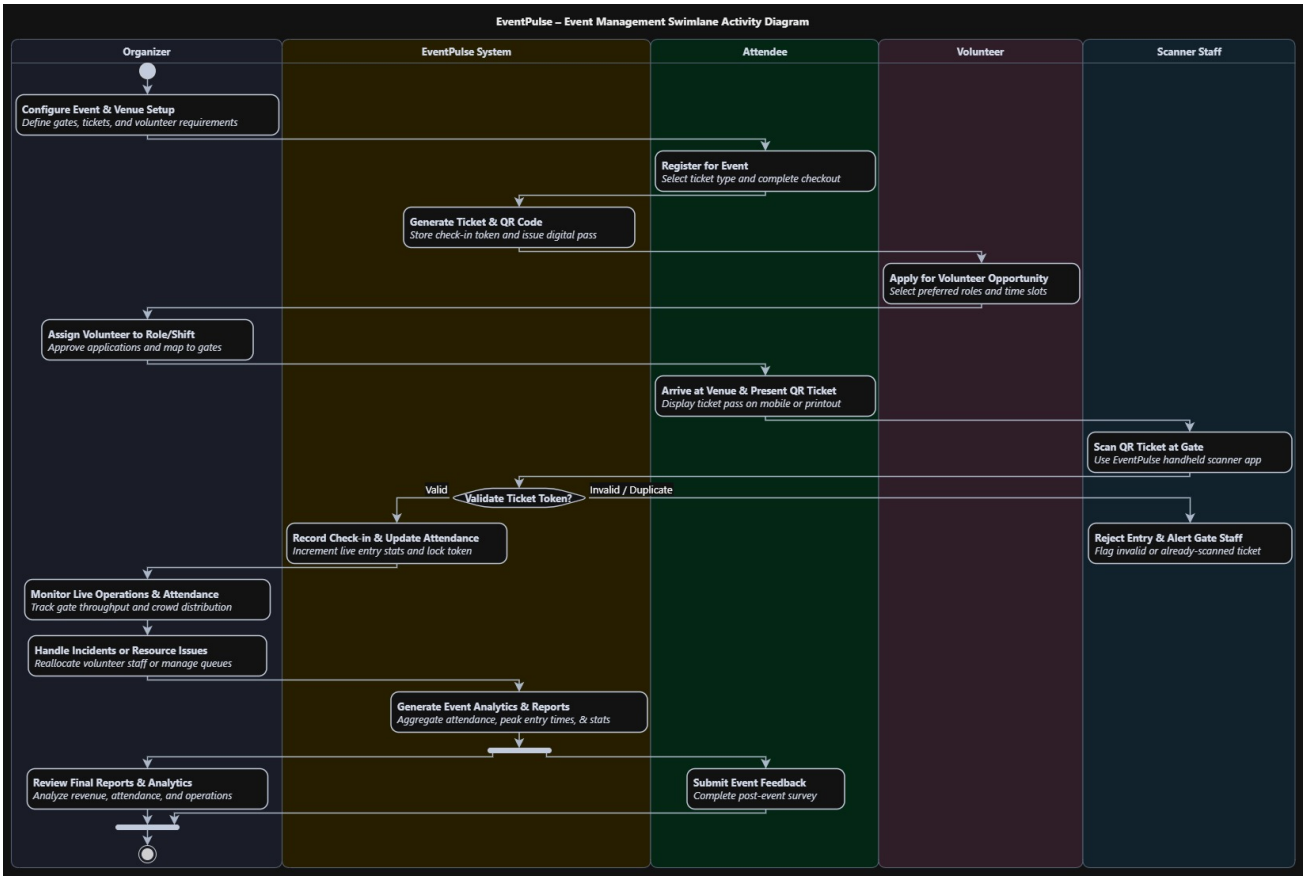



**Figure 2.2:** EventPulse Event Operations Activity/Swimlane Diagram

# 3 Results and Evaluation

## 3.1 Testing Methodology

Frontend UI testing is complete; the remaining layers are planned for the backend stage.

**Table 3.1:** Testing Categories

| Category | Scope |
|---|---|
| **Frontend UI** | Navigation, forms, ticket display, dashboards, scanner views, responsiveness. |
| **Unit** | Ticket validation, state transitions, transfer, attendance, volunteer assignment. |
| **Integration** | Frontend–backend, backend–database, analytics, real-time, scanner–validation. |
| **Security** | Unauthorized access, invalid, expired, or duplicate QR codes, invalid transfers. |
| **Concurrency** | Simultaneous multi-gate scans must never accept one ticket twice. |
| **Performance and Analytics** | Validation latency under load; MAE, RMSE, precision, and recall for models. |

### 3.2 Performance Metrics

**Table 3.2:** EventPulse Evaluation Metrics

| Metric | Target |
|---|---|
| QR validation time | Less than 2 seconds |
| Duplicate scan / invalid ticket rejection | 100% rejection |
| Concurrent check-ins | No inconsistent ticket states |
| Dashboard update time | Near real-time |
| Ticket transfer | 100% old-QR invalidation |
| Volunteer assignment | Correct role and shift mapping |
| Prediction error | Model-dependent |

### 3.3 Analysis

Server-side validation and the `ISSUED` → `CHECKED_IN` state give central control over entry and prevent duplicates, while transfer invalidation keeps ownership secure. The organizer dashboard and Volunteer Hub reduce tool switching and informal coordination. Analytics results on simulated data are feasibility demonstrations, not validated predictions. Realistic large-scale event datasets are unavailable; a pilot with about 20–50 attendees across multiple gates is proposed.

## 4 Conclusion

### 4.1 Summary of Work

EventPulse unifies event creation, ticketing, QR check-in, ticket transfer, volunteer coordination, monitoring, analytics, and reporting. The React/TypeScript/Tailwind frontend is complete; the backend and data infrastructure are under development.

### 4.2 Key Findings

- **Unified management** reduces fragmentation across event activities.
- **Server-side validation and ticket states** control entry and block duplicates.
- **Transfer invalidation** strengthens ticket ownership control.
- **Central dashboards and structured volunteer roles** improve visibility and coordination.

### 4.3 Future Work and Recommendations

Complete the NestJS backend and PostgreSQL integration; add Socket.IO live updates and Redis/BullMQ background jobs; develop and evaluate predictive models; add offline scanning with SQLite; extend to crowd-flow and cross-event analytics; and run scalability and pilot testing.

### 4.4 Final Remarks

EventPulse moves event management from fragmented manual workflows to a centralized operational platform. With backend integration, testing, and evaluation, it can scale to medium-sized college events, conferences, and hackathons.

# Bibliography

[1] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.

[2] ISO/IEC 18004, *Information technology – Automatic identification and data capture techniques – QR Code bar code symbology specification*. International Organization for Standardization.

[3] Meta Open Source, "React Documentation." `https://react.dev`

[4] Microsoft, "TypeScript Documentation." `https://www.typescriptlang.org/docs/`

[5] Tailwind Labs, "Tailwind CSS Documentation." `https://tailwindcss.com/docs`

[6] NestJS, "NestJS Documentation." `https://docs.nestjs.com`

[7] The PostgreSQL Global Development Group, "PostgreSQL Documentation." `https://www.postgresql.org/docs/`

[8] The pandas development team, "pandas Documentation." `https://pandas.pydata.org/docs/`

[9] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[10] GitHub, "GitHub Actions Documentation." `https://docs.github.com/actions`